

Introduction

In this practical we were asked to implement a renderer in WebGL.

Part 1

For part 1 I created a regular icosahedron to use as the demonstration object. Defining objects in code is quite laborious (and makes the code less readable), so I've decided to implement the object loader right away. This way I can define the object in a separate file and simply load it into the scene. A detailed description of the object loader is given in part 4.

The object has a texture¹ applied to it. To keep track of the textures, we keep a global list, each element of which contains a texture's buffer, index, and `gl.TEXTUREn` index. We can then select any texture out of this list and attach it to an object. The draw function will then bind whatever texture we've attached to the shader.

I've also implemented a movable camera to make viewing the scene a bit easier. There are 2 globals that keep track of camera position: `camera_rotation` and `camera_distance`. The `update_projection_mx` function can be called to recalculate the view matrix based on the values of these 2 globals.

The oblique projection is implemented as an orthographic projection, which is then sheared along the x and y (viewing) axes. The effect achieved by this is similar to rotating the camera.

To animate the object, I assigned a function to the `object.animation` attribute. This function will be called on every animation frame and will apply a transform to the object's model matrix. This method allows us to easily define any animation we want for each object.

Part 2

This part builds on top of part 1 by implementing perspective projection, adding more objects, animations, and a directional light source.

Perspective projection is pretty straightforward — we create a projection matrix using `glMatrix` with an FOV of 45°.

Since we keep our textures in a list, to apply different textures to different objects, we just set `object.texture` to the desired texture object.

The directional light is implemented by defining a vector pointing towards the light. We can then take the dot product of a vertex's normal and the light direction to get how much that point should be lit. Then we just multiply this coefficient with a color of the directional light (I chose [0.75, 0.75, 0.75]).

The shearing animation is implemented similarly to the rotation from part 1.

¹Textures used in this practical are CC0, taken from <https://cc0-textures.com/>

Part 3 - Phong shading (without the specular component) with normal maps

Since normal maps are in no way different from textures, it's possible to load them as textures using the same mechanism as in parts 1 and 2. However, instead of assigning the map to `object.texture` we will assign it to the `object.normal_map` attribute. The draw function will then take the texture attribute and bind it to `uNormalMap` uniform variable.

To apply a normal map to an object we'll need to transform it so that the normals point in the correct direction on each face. To do this, we will need to calculate tangent and bitangent vectors for each point²³. Since we've happened to have implemented an object loader, we can calculate the vectors as the last step of object import.

With the vectors calculated, we can use them to build a matrix that will allow us to transform tangent space coordinates of the normal map into the world space coordinates that we use for lighting. We will do that in the vertex shader and then pass the matrix to the fragment shader².

The fragment shader will sample the normal map provided by `uNormalMap` and multiply it by the translation matrix, producing a normal vector of the fragment in world space. This vector can then be used for calculating the directional component of the Phong shading model.

Part 4 - Object loader

To demonstrate the object loader's capability, `part4.html` loads a third-party model⁴ consisting of 60k triangles. A texture and a normal map are applied to this the object to further demonstrate the shading from part 3.

The object loader works by loading the `.obj` file using the `fetchAPI` and parsing it line by line.

The loader supports the following keywords:

- `v` (vertex)
- `vn` (vertex normal)
- `vt` (vertex texcoord)
- `f` (face information)

²This practical's implementation of normal mapping was based off of the technique described in:

<https://learnopengl.com/Advanced-Lighting/Normal-Mapping>

³Another tutorial explaining normal mapping that was also referenced:

<https://apoorvaj.io/exploring-bump-mapping-with-webgl>

⁴Hydria apothecary vase by Virtual Museums of Małopolska, available at:

<https://sketchfab.com/3d-models/hydria-apothecary-vase-7d6938c0c0b54b06a0210a982a73023e>

When the parser encounters a vertex, the vertex is added to the object's vertex list.

When the parser encounters a normal or a texture coordinate, it is recorded into a separate list (not the object's), as the order of normals and texcoords does not necessarily correspond to the order of vertices, and so they'll need to be reordered based on the face data.

When the parser encounters a face, the chunks following the "f" keyword are assumed to contain triplets of `vertex_index/texcoord_index/normal_index`. We then go through the triplets and assign the values of texture coordinates and normals we recorded to `object.texcoords` and `object.normals` so that their positions in the list correspond to positions of vertices in `object.vertices`. Finally, we will calculate the tangent and bitangent vectors, as they are required for part 3 to work.

Once the end of the file is reached, the resulting object is returned.

Since the `fetch` function is used, `loadObjFile` will return a promise of an object, not the object itself. So to load the object into a scene, we will need to pass it into the `addObjectToScene` function. This function will register the object provided to it with `objectData` and `objectBuffers` globals, which are the two lists from which the `render` function takes information about the objects that should be rendered. `addObjectToScene` will return the object so that we can chain another function to it should we wish to apply any modifications to the object, such as assigning a texture, applying a transform, etc.